UCS420: Cognitive Computing

Assignment 2 — Python Data Structures

Name: Krish Marwaha
Roll Number: 1024170223
Branch: CSE

Solved Assignment

Simple Python solutions with the required comments and explanations.

Question 1 — Lists

```
# Q1: Lists
roll_no = 1024170223
digits = [int(d) for d in str(roll_no)]
L = [d * 10 for d in digits]

print("i. L =", L)

L.append(50)  # append() adds 50 at the end
print("ii. After append(50):", L)

L.insert(2, 25)  # insert() adds 25 at index 2
print("After insert(2, 25):", L)

L.remove(50)  # remove() removes the first occurrence of 50
print("iii. After remove(50):", L)

removed = L.pop(0)  # pop(0) removes and returns the element at index 0
print("After pop(0):", L)

L.sort()
print("iv. Ascending:", L)

L.sort(reverse=True)
print("Descending:", L)

print("v. First three:", L[:3])
print("Last three:", L[-3:])

average = sum(L) / len(L)
above_average = [x for x in L if x > average]
print("vi. Elements greater than average:", above_average)
```

Expected key values: Initial L = [10, 0, 20, 40, 10, 70, 0, 20, 20, 30]; after append = [10, 0, 20, 40, 10, 70, 0, 20, 20, 30, 50]; after insert = [10, 0, 25, 20, 40, 10, 70, 0, 20, 20, 30, 50]; after removals = [0, 25, 20, 40, 10, 70, 0, 20, 20, 30]; ascending = [0, 0, 10, 20, 20, 20, 25, 30, 40, 70]; descending = [70, 40, 30, 25, 20, 20, 20, 10, 0, 0]; average = 23.50; above average = [25, 40, 70, 30].

Question 2 — Tuples

```
# Q2: Tuples
scores = tuple(L[:8])

highest = max(scores)
print("i. Highest score:", highest)
print("Index of highest:", scores.index(highest))

lowest = min(scores)
print("Lowest score:", lowest)
print("Times lowest appears:", scores.count(lowest))

# Tuples are immutable, so they cannot be reversed in place.
reversed_scores = list(reversed(scores))
print("ii. Reversed tuple as list:", reversed_scores)

score = int(input("iii. Enter a score: "))
if score in scores:
    print("First occurrence index:", scores.index(score))
else:
    print("Score not present")

try:
    scores[0] = 100
except TypeError as e:
    print("iv. Error:", e)
    # Tuples cannot be changed after creation, unlike lists.

first, second, *remaining = scores
print("v. First:", first, "Second:", second, "Remaining:", remaining)
```

scores: (10, 0, 20, 40, 10, 70, 0, 20). Highest = 70 at index 5. Lowest = 0, appearing 2 time(s). Reversed list = [20, 0, 70, 10, 40, 20, 0, 10].

The attempted assignment to a tuple element raises TypeError because tuples are immutable; list elements can be changed.

Question 3 — Random Numbers

```
# Q3: Sets / Random Numbers
import random
import math
from collections import Counter

random.seed(1024170223)
numbers = [random.randint(100, 900) for _ in range(100)]

odd_numbers = [x for x in numbers if x % 2 != 0]
even_numbers = [x for x in numbers if x % 2 == 0]

def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

prime_numbers = [x for x in numbers if is_prime(x)]

print("i. Random list:", numbers)
print("ii. Odd count:", len(odd_numbers))
print("Odd numbers:", odd_numbers)
print("iii. Even count:", len(even_numbers))
print("Even numbers:", even_numbers)
print("iv. Prime count:", len(prime_numbers))
print("Prime numbers:", prime_numbers)

counts = Counter(numbers)
number, frequency = counts.most_common(1)[0]
print("v. Most frequent number:", number)
print("Frequency:", frequency)
```

Seed: 1024170223. Odd count = 49, even count = 51, prime count = 21. Most frequent number = 776, occurring 2 time(s).

Question 4 — Sets

```
# Q4: Sets
digits = [int(d) for d in str(1024170223)][:8]

A = {d * 7 for d in digits}
B = {d * 9 for d in digits}

print("A =", A)
print("B =", B)

print("vi. Union:", A.union(B))
print("vii. Intersection:", A.intersection(B))

print("viii. A - B:", A.difference(B))
print("B - A:", B.difference(A))
# difference() gives one-directional difference; symmetric_difference() gives values in either set but not both.

print("ix. Symmetric difference:", A.symmetric_difference(B))

print("x. A is subset of B:", A.issubset(B))
print("B is superset of A:", B.issuperset(A))

X = int(input("xi. Enter X: "))
A.discard(X)
print("A after discard:", A)
# discard() does not raise an error if X is not present, unlike remove().
```

A: [0, 7, 14, 28, 49]

B: [0, 9, 18, 36, 63]

Union: [0, 7, 9, 14, 18, 28, 36, 49, 63]

Intersection: [0]

A – **B:** [7, 14, 28, 49]

B – **A:** [9, 18, 36, 63]

Symmetric difference: [7, 9, 14, 18, 28, 36, 49, 63]

A subset B: False; **B superset A:** False.

Question 5 — Dictionaries

```
# Q5: Dictionaries
my_dict = {
    "name": "Krish Marwaha",
    "roll_no": "1024170223",
    "branch": "CSE",
    "age": 20,
    "city": "Amritsar"
}

# i. Rename city to location using pop()
my_dict["location"] = my_dict.pop("city")
print("i.", my_dict)

# ii. Add CGPA
my_dict["cgpa"] = 8.47
print("ii.", my_dict)

# iii. Increase age by 1
my_dict["age"] += 1
print("iii.", my_dict)

# iv. Two separate copies
dict1 = my_dict.copy()
removed = dict1.pop("branch")
print("Using pop():", dict1)

dict2 = my_dict.copy()
del dict2["branch"]
print("Using del:", dict2)

# pop() returns the deleted value, while del only deletes it.

# v. Iterate using items()
for key, value in my_dict.items():
    print(key, "→", value)

# vi. Safe email check
if "email" in my_dict:
    print(my_dict["email"])
else:
    print("Email key does not exist.")

# vii. Merge with a fictional friend's dictionary
friend_dict = {
    "name": "Aman Sharma",
    "roll_no": "1024170999",
    "branch": "CSE",
    "age": 20,
    "city": "Delhi"
}

merged = {**my_dict, **friend_dict}
print("Merged dictionary:", merged)

# If keys are common, values from friend_dict win because it is unpacked second.

# viii. Keep only string values
string_values = {k: v for k, v in my_dict.items() if isinstance(v, str)}
print("String values only:", string_values)
```

Final original dictionary: {'name': 'Krish Marwaha', 'roll_no': '1024170223', 'branch': 'CSE', 'age': 21, 'location': 'Amritsar', 'cgpa': 8.47}
String-only dictionary: {'name': 'Krish Marwaha', 'roll_no': '1024170223', 'branch': 'CSE', 'location': 'Amritsar'}